



RECOVERY PROCEDURE

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Windows Update Crash Recovery

Complete recovery runbook for unplanned host reboot caused by Windows Update, covering vSAN, NSX, SDDC Manager, and full lab restoration.

CRASH RECOVERY

WINDOWS UPDATE

VCF

PROCEDURE

VCF 9.0

VMware Cloud Foundation

VCF 9.0 Lab Disaster Recovery — Windows Update Crash

Where commands run in this runbook. Recovery touches three environments:

- **Windows workstation (PowerShell as Administrator)** — managing VMware Workstation VM lifecycle (`vmrun` , `Get-VM` via Workstation), Windows Update history, scheduled task disabling, system restore. Run elevated.
- **VMware Workstation UI** — power-on order, VM repair, snapshot management. GUI-only operations called out in narrative steps.
- **ESXi host shell (SSH)** — once the nested ESXi VMs are back online, vSAN partition recovery, network re-binding, datastore remount via `esxcli` , `cmds-tool` .

A few steps run from **vCenter UI (browser)** for cluster-level recovery actions. Replace any credential placeholder like `<your-password>` with values from your vault.

Complete Recovery Runbook

Version: 1.1 **Date:** May 2026 **Environment:** VMware Workstation 17 Nested Lab **Incident:** Unplanned host reboot caused by Windows Updates on March 13–15, 2026

PowerShell users: Every bash block in this document that targets a REST API or other client-runnable command has a paste-ready PowerShell sibling block underneath it. For the full substitution table — `curl` → `Invoke-RestMethod` , base64 auth headers, `here-docs` → `here-strings` , `jq` / `python3 -m json.tool` → `ConvertFrom-Json` / `ConvertTo-Json -Depth N` , etc. — see [POWERSHELL-TRANSLATION-REFERENCE.md](#). Blocks targeting the appliance shell only (`psql` , `systemctl` , `vi` , ESXi `esxcli` / `esxcfg-vswitch` / `vmkping` , NSX admin CLI) are flagged "Run on the appliance only" with no PS sibling — they cannot run from a Windows workstation.

Table of Contents

1. [Incident Summary](#)
2. [Environment Reference](#)
3. [Phase 1: vSAN Cluster Recovery](#) - 3.1 [Symptoms](#) - 3.2 [Root Cause](#) - 3.3 [vSAN Partition Diagnosis](#) - 3.4 [Resolution: vSwitch Failover Policy Fix](#) - 3.5 [vSAN Object Resync Verification](#)
4. [Phase 2: NSX Manager Recovery](#) - 4.1 [Symptoms](#) - 4.2 [Root Cause](#) - 4.3 [Resolution: NSX Service Restart](#) - 4.4 [NSX Verification](#)
5. [Phase 3: SDDC Manager Recovery](#) - 5.1 [CPU Soft Lockup — First Occurrence](#) - 5.2 [Hard Reset via vCenter API](#) - 5.3 [Waiting for Spring Boot Services](#) - 5.4 [Service Health Verification](#)
6. [Phase 4: VCF Management Components Deployment](#) - 6.1 [Deployment Task Status](#) - 6.2 [Fleet \(vRSLCM\) CPU Soft Lockup](#) - 6.3 [Resource Contention Mitigation](#) - 6.4 [Fleet Recovery and Database Investigation](#) - 6.5 [SDDC](#)

[Manager Task Retry and Completion](#)

7. [Phase 5: VCF Operations Cluster Initialization](#) - 7.1 [INITIALIZATION_FAILED State](#) - 7.2 [HSQLDB Reset Procedure](#) - 7.3 [Admin Password Reset](#) - 7.4 [SHA1 Thumbprint Fix](#) - 7.5 [Successful Cluster Initialization](#)
 8. [Phase 6: VCF Operations Admin Role Fix](#) - 8.1 [Symptoms](#) - 8.2 [Investigation](#) - 8.3 [Resolution: Role Assignment via Suite-API](#) - 8.4 [Super Admin Behavior \(By Design\)](#)
 9. [Phase 7: VCF Operations Adapter Configuration](#) - 9.1 [Collector VM Recovery](#) - 9.2 [Adapter Restart Procedure](#) - 9.3 [NSX Adapter Manual Creation](#) - 9.4 [Final Adapter Status](#)
 10. [Key Learnings and Reference](#)
 - 10.1 [Critical API Endpoints Used](#)
 - 10.2 [Service Startup Times Under Load](#)
 - 10.3 [Common Pitfalls](#)
 11. [Document Information](#)
-

1. Incident Summary

On March 13, 2026, the Windows host running the nested VCF 9.0 lab environment was force-rebooted by Windows Updates. This caused an unclean shutdown of all nested VMs simultaneously, including all four ESXi hosts, vCenter, SDDC Manager, NSX Manager, and the VCF management components that were in the process of being deployed.

Impact: - All VMs powered off ungracefully - vSAN cluster entered a partitioned state - NSX Manager services became unstable - SDDC Manager experienced CPU soft lockups from resource contention - VCF Management Components deployment task (Fleet, VCF Operations, Collector) was interrupted mid-deploy at step 25 of 28 - Fleet (vRSLCM) experienced CPU soft lockups - VCF Operations cluster was stuck in `INITIALIZATION_FAILED` state

Recovery Duration: Approximately 48 hours across multiple troubleshooting sessions

Outcome: Full recovery achieved — all VCF components operational

2. Environment Reference

2.1 VM Inventory (Post-Deployment)

Component	Hostname	IP Address	VM ID	vCPU	RAM
ESXi Host 1	esxi01.lab.local	192.168.1.201	—	8	48 GB
ESXi Host 2	esxi02.lab.local	192.168.1.202	—	8	48 GB
ESXi Host 3	esxi03.lab.local	192.168.1.203	—	8	48 GB
ESXi Host 4	esxi04.lab.local	192.168.1.204	—	8	48 GB
vCenter Server	vcenter.lab.local	192.168.1.69	vm-18	2	16 GB
SDDC Manager	sddc-manager.lab.local	192.168.1.241	vm-68	4	16 GB
NSX Manager	nsx-manager.lab.local	192.168.1.71	vm-58	6	30 GB
NSX VIP	nsx-vip.lab.local	192.168.1.70	—	—	—

Component	Hostname	IP Address	VM ID	vCPU	RAM
Fleet (vRSLCM)	fleet.lab.local	192.168.1.78	vm-4014	4	12 GB
VCF Operations	vcf-ops.lab.local	192.168.1.77	vm-4015	8	32 GB
Collector	collector.lab.local	192.168.1.79	vm-4016	4	16 GB
Logs	—	—	vm-69	4	8 GB

Total nested VM resources: 32 vCPU, 130 GB RAM (management VMs only, excluding ESXi hosts)

2.2 Credentials

Component	Username	Password	Auth Method
ESXi Hosts	root		SSH
vCenter	administrator@vsphere.local		API / SSO
SDDC Manager SSH	vcf		SSH
SDDC Manager API	administrator@vsphere.local		REST API
NSX Manager	admin / root		SSH / API
Fleet API	admin@local		REST API
VCF Operations	admin		Suite-API (local)
VCF Operations SSH	root		SSH

2.3 Key IDs and References

Item	ID
SDDC Manager UUID	90ffb005-52c9-4d35-b254-0217f5305b59
Fleet Environment ID	df6d02bb-692a-4c44-a0d3-99e29c672bd0
Fleet Request ID	be0221fd-e620-48f3-8543-eb67b26616b0
Deployment Task ID	a48065d5-1ead-48ea-9d1e-113ae80732d2
VCF Ops Admin User ID	6df57f67-9573-47a8-a9d4-e9efa841a2ba
vCenter GUID	92109cf0-ad3b-4ffa-8972-a77bb7fadacf
NSX Cluster ID	6c55d856-ab96-4190-8495-3cc8cb23450c

3. Phase 1: vSAN Cluster Recovery

3.1 Symptoms

After the Windows host rebooted, the vSAN datastore was inaccessible and the vSAN cluster showed a partitioned state:

- vSAN health showed "Cluster partition detected"
- VMs on the vSAN datastore were unresponsive
- `esxcli vsan cluster get` showed hosts in separate sub-clusters
- vmkping between hosts on the vSAN network (192.168.12.x) failed

3.2 Root Cause

The ungraceful shutdown caused the vSAN VMkernel network configuration to revert or become inconsistent. The vSwitch failover policies for the vSAN portgroup had reverted to using an incorrect NIC teaming configuration, preventing vSAN traffic between hosts.

3.3 vSAN Partition Diagnosis

Step 1: Check vSAN cluster membership on each host:

```
# Run on the appliance only (ESXi shell via SSH – no client equivalent)
# SSH to each ESXi host
esxcli vsan cluster get
```

Step 2: Verify vSAN VMkernel connectivity:

```
# Run on the appliance only (ESXi shell – vmkping uses VMkernel stack)
# From esxi01, ping other hosts on vSAN network
vmkping -I vmk2 192.168.12.75
vmkping -I vmk2 192.168.12.76
vmkping -I vmk2 192.168.12.82
```

Step 3: Check vSwitch NIC teaming:

```
# Run on the appliance only (ESXi shell – esxcli must run on the host)
esxcli network vswitch standard policy failover get --vswitch-name=vSwitch0
```

3.4 Resolution: vSwitch Failover Policy Fix

The vSAN portgroup failover policy needed to be corrected on all four ESXi hosts:

```
# Run on the appliance only (ESXi shell – esxcfg-vswitch only exists on ESXi)
# Fix the failover policy for the vSAN VMkernel portgroup
esxcfg-vswitch -p "vSAN" -N vmnic0 vSwitch0
```

After correcting the failover policy on all hosts, vSAN traffic resumed and the cluster reformed.

3.5 vSAN Object Resync Verification

```
# Run on the appliance only (ESXi shell – esxcli vsan must run on a host)
# Monitor vSAN resync progress
esxcli vsan debug resync summary get

# Verify cluster health in vCenter
# Navigate to: Cluster > Monitor > vSAN > Health
```

Note: vSAN object resync took approximately 30-45 minutes after the cluster reformed. All objects returned to a compliant state.

4. Phase 2: NSX Manager Recovery

4.1 Symptoms

After vSAN recovery, NSX Manager was reachable but unstable: - NSX Manager UI intermittently available - SDDC Manager reported NSX status as "UNSTABLE" - NSX services were in a crash loop

4.2 Root Cause

The ungraceful shutdown corrupted some NSX service state. The NSX Manager services needed a clean restart to re-establish their internal cluster state.

4.3 Resolution: NSX Service Restart

Step 1: SSH to NSX Manager and check service status:

```
# Run on the appliance only (NSX admin CLI – these verbs only exist inside NSX shell)
ssh admin@192.168.1.71
get service
```

Step 2: Restart critical services:

```
# Run on the appliance only (NSX admin CLI inside `ssh admin@<nsx>`)
restart service manager
restart service proton
restart service corfu
```

Step 3: Wait for services to stabilize (5-10 minutes):

```
# Run on the appliance only (NSX admin CLI)
get cluster status
```

4.4 NSX Verification

```
# bash on any Linux client (or SDDC Manager via SSH)
# Verify via SDDC Manager API
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – requires $TOKEN from earlier auth call
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
$h = @{ Authorization = "Bearer $TOKEN" }
Invoke-RestMethod -Uri 'https://sddc-manager.lab.local/v1/nsxt-clusters' -Headers $h |
  ConvertTo-Json -Depth 10
```

Expected output: "status": "ACTIVE"

5. Phase 3: SDDC Manager Recovery

5.1 CPU Soft Lockup — First Occurrence

Symptoms observed on VM console:

```
watchdog: BUG: soft lockup - CPU#0 stuck for 22s! [java:12345]
```

The SDDC Manager VM console showed a CPU soft lockup, indicating the Java-based services (Spring Boot) were consuming all available CPU and the Linux kernel scheduler couldn't run other processes.

Root Cause: Resource contention — with all management VMs running simultaneously (32 vCPU, 130 GB RAM in nested VMs), the physical host couldn't provide enough CPU time to all VMs. SDDC Manager's `domainmanager` service (Spring Boot) is particularly CPU-intensive during startup.

5.2 Hard Reset via vCenter API

Since SSH was unresponsive due to the soft lockup, the VM had to be hard-reset through the vCenter REST API:

```
# bash on any Linux client (or SDDC Manager via SSH) – vCenter REST is reachable from anywhere
# Get vCenter API session
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -H "Authorization: Basic $(echo -n 'administrator@vsphere.local:<your-password>' | base64)")

# Hard reset the SDDC Manager VM
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-68/power?action=reset" \
  -H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
```

```

$vc = 'vcenter.lab.local'
$user = 'administrator@vsphere.local'
$pass = '<your-password>'

# Get vCenter API session (Basic auth, returns the session-id as a JSON string)
$pair = "{0}:{1}" -f $user, $pass
$basic = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$SESSION = Invoke-RestMethod -Method Post -Uri "https://$vc/api/session" `
    -Headers @{ Authorization = "Basic $basic" }

# Hard reset the SDDC Manager VM
Invoke-RestMethod -Method Post `
    -Uri "https://$vc/api/vcenter/vm/vm-68/power?action=reset" `
    -Headers @{ 'vmware-api-session-id' = $SESSION }

```

Warning: Hard reset is destructive and should only be used when SSH and console are completely unresponsive due to soft lockups. Always prefer graceful restart first.

5.3 Waiting for Spring Boot Services

After hard reset, the SDDC Manager takes a significant amount of time to fully start under resource contention:

Service	Port	Expected Start Time	Actual Start Time (Under Load)
domainmanager	7200	2-3 minutes	~37 minutes
operationsmanager	7300	2-3 minutes	~30 minutes
lcm	7400	2-3 minutes	~25 minutes

Monitoring startup progress:

```

# Run on the appliance only (SDDC Manager via SSH - `ssh`, `systemctl`, and `localhost` only resolve there)
# SSH to SDDC Manager (once responsive)
ssh vcf@192.168.1.241

# Check if domainmanager port is bound
ss -tlnp | grep 7200

# Check service status
systemctl status domainmanager
systemctl status operationsmanager

# Watch SDDC Manager API health
curl -sk https://localhost/v1/system/health

```

Critical Note: The `domainmanager` service uses HTTP on port 7200 (not HTTPS). Using `curl -sk https://localhost:7200` will fail with "wrong version number". Always use `http://localhost:7200` for direct service health checks.

5.4 Service Health Verification

```
# Run on the appliance for `systemctl`; the final `curl` works from any client
# Verify all SDDC Manager services are running (appliance-only)
systemctl list-units --type=service --state=running | grep -E 'domain|operations|lcm|common'

# Expected output should show:
# domainmanager.service      loaded active running
# operationsmanager.service  loaded active running
# lcm.service                 loaded active running
# commonsvcs.service          loaded active running

# Verify API is responsive (any client)
curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<your-password>"}' \
  | python3 -c "import sys,json; t=json.load(sys.stdin); print('Token:', t['accessToken'][:20]+'...')"
```

PowerShell equivalent for the API health check (run on your Windows workstation):

```
# PowerShell on Windows workstation – `systemctl` has no Windows equivalent
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$body = @({
    username = 'administrator@vsphere.local'
    password = '<your-password>'
} | ConvertTo-Json)
$resp = Invoke-RestMethod -Method Post `
  -Uri 'https://sddc-manager.lab.local/v1/tokens' `
  -ContentType 'application/json' -Body $body
"Token: $($resp.accessToken.Substring(0,20))..."
```

6. Phase 4: VCF Management Components Deployment

6.1 Deployment Task Status

The VCF Management Components deployment (Fleet, VCF Operations, Collector) was interrupted at step 25 of 28 when the Windows crash occurred.

Checking deployment status:

```
# bash on any Linux client (or SDDC Manager via SSH)
# Get auth token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<your-password>"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")

# Check management components status
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/vcf-management-components | python3 -m json.tool
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$sddc = 'sddc-manager.lab.local'
$authBody = @{
    username = 'administrator@vsphere.local'
    password = '<your-password>'
} | ConvertTo-Json
$auth = Invoke-RestMethod -Method Post -Uri "https://$sddc/v1/tokens" `
    -ContentType 'application/json' -Body $authBody
$TOKEN = $auth.accessToken
$h = @{ Authorization = "Bearer $TOKEN" }

# Check management components status
Invoke-RestMethod -Uri "https://$sddc/v1/vcf-management-components" -Headers $h |
    ConvertTo-Json -Depth 10
```

Task status showed: - Steps 1-25: Successful - Step 26: FAILED — "Retrieve the status for VCF Operations with VCF Operations collector Deployment request" - Unable to get request status from Fleet for request ID `be0221fd-e620-48f3-8543-eb67b26616b0` - Steps 27-28: Not attempted

6.2 Fleet (vRSLCM) CPU Soft Lockup

Fleet (vm-4014) also experienced a CPU soft lockup:

Symptoms:

```
watchdog: BUG: soft lockup - CPU#0 stuck for 22s! [java:5678]
```

Resolution — Hard reset via vCenter API:

```
# bash on any Linux client – reuses $SESSION from the vCenter auth call earlier
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4014/power?action=reset" \
    -H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $SESSION variable from earlier vCenter auth block
Invoke-RestMethod -Method Post `
    -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-4014/power?action=reset' `
    -Headers @{ 'vmware-api-session-id' = $SESSION }
```

Fleet startup time: Port 8080 took approximately **48 minutes** to become available after hard reset, due to: 1. Java LCM backend initialization 2. PostgreSQL database recovery 3. Embedded Nginx reverse proxy startup

6.3 Resource Contention Mitigation

With all management VMs running, the total resource demand (32 vCPU, 130 GB RAM) caused severe contention. The solution was to temporarily power off non-essential VMs:

```
# bash on any Linux client – reuses $SESSION from the vCenter auth call earlier
# Power off Collector VM (already crashed)
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?action=stop" \
```

```
-H "vmware-api-session-id: $SESSION"

# Power off Logs VM (not needed for recovery)
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-69/power?action=stop" \
-H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $SESSION variable from earlier vCenter auth block
$h = @{ 'vmware-api-session-id' = $SESSION }

# Power off Collector VM (already crashed)
Invoke-RestMethod -Method Post `
  -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?action=stop' -Headers $h

# Power off Logs VM (not needed for recovery)
Invoke-RestMethod -Method Post `
  -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-69/power?action=stop' -Headers $h
```

Resources freed: 8 vCPU + 24 GB RAM

Lesson Learned: In nested environments with limited resources, prioritize which VMs need to run simultaneously. Power off non-essential VMs during recovery to prevent CPU soft lockups.

6.4 Fleet Recovery and Database Investigation

After Fleet came back online, its API was returning HTTP 500 errors for the deployment request. Investigation through the PostgreSQL database revealed the request had actually completed:

Step 1: SSH to Fleet and query the database:

```
# Run on the appliance only (psql + sudo only exist on Fleet's Photon shell)
ssh root@192.168.1.78

# Connect to the vrlcm PostgreSQL database
sudo -u postgres psql -d vrlcm
```

Step 2: Check the request status:

```
SELECT id, state, requesttype, created, completed
FROM vm_rs_request
WHERE id = 'be0221fd-e620-48f3-8543-eb67b26616b0';
```

Result: The request was already in `COMPLETED` state — Fleet's own crash recovery had processed it during its long startup sequence.

Step 3: Verify via Fleet API:

```
# bash on any Linux client (or Fleet appliance via SSH)
# Fleet auth
curl -sk -X POST https://fleet.lab.local:8080/lcm/authzn/api/login \
-H "Content-Type: application/json" \
-d '{"username":"admin@local","password":"<your-password>"}'

# Check environment status
```

```
curl -sk -H "Authorization: Bearer $TOKEN" \
https://fleet.lab.local:8080/lcm/lcops/api/v2/environments/df6d02bb-692a-4c44-a0d3-99e29c672bd0
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

# Fleet auth – note: returns the token in a JSON body or in an `x-xenon-auth-token` response header
$loginBody = @{ username = 'admin@local'; password = '<your-password>' } | ConvertTo-Json
$loginResp = Invoke-WebRequest -Method Post `
    -Uri 'https://fleet.lab.local:8080/lcm/authzn/api/login' `
    -ContentType 'application/json' -Body $loginBody
# Token may live in either the response header or body – verify in your environment
$TOKEN = $loginResp.Headers['x-xenon-auth-token']
if (-not $TOKEN) { $TOKEN = ($loginResp.Content | ConvertFrom-Json).token }

# Check environment status
$envId = 'df6d02bb-692a-4c44-a0d3-99e29c672bd0'
Invoke-RestMethod `
    -Uri "https://fleet.lab.local:8080/lcm/lcops/api/v2/environments/$envId" `
    -Headers @{ Authorization = "Bearer $TOKEN" } |
    ConvertTo-Json -Depth 10
```

Uncertainty flag: Fleet (vRSLCM) returns its auth token in different places depending on version — sometimes the body's `token` / `accessToken` field, sometimes the `x-xenon-auth-token` response header. Verify which your environment uses before scripting.

6.5 SDDC Manager Task Retry and Completion

With Fleet reporting the request as completed, the SDDC Manager deployment task needed to be retried:

Retry the stuck task:

```
# bash on any Linux client (or SDDC Manager via SSH)
curl -sk -X PATCH "https://sddc-manager.lab.local/v1/tasks/a48065d5-1ead-48ea-9d1e-113ae80732d2" \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"status":"IN_PROGRESS"}
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $TOKEN from earlier SDDC Manager auth
$taskId = 'a48065d5-1ead-48ea-9d1e-113ae80732d2'
$retryBody = @{ status = 'IN_PROGRESS' } | ConvertTo-Json
Invoke-RestMethod -Method Patch `
    -Uri "https://sddc-manager.lab.local/v1/tasks/$taskId" `
    -Headers @{ Authorization = "Bearer $TOKEN" } `
    -ContentType 'application/json' -Body $retryBody
```

Result: After approximately 60 seconds, the task progressed through steps 26, 27, and 28 — all successful.

Final status: 28/28 subtasks completed successfully.

Verification:

```
# bash on any Linux client (or SDDC Manager via SSH)
# Confirm management components status
curl -sk -H "Authorization: Bearer $TOKEN" \
https://sddc-manager.lab.local/v1/vcf-management-components | python3 -m json.tool
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $TOKEN from earlier SDDC Manager auth
Invoke-RestMethod -Uri 'https://sddc-manager.lab.local/v1/vcf-management-components' `
-Headers @{ Authorization = "Bearer $TOKEN" } |
ConvertTo-Json -Depth 10
```

Expected:

```
{
  "vcfOperationsFleetManagement": "SUCCEEDED",
  "vcfOperations": "SUCCEEDED",
  "vcfOperationsCollector": "SUCCEEDED"
}
```

7. Phase 5: VCF Operations Cluster Initialization

7.1 INITIALIZATION_FAILED State

After the VMs were recovered, VCF Operations (vcf-ops.lab.local) was stuck in `INITIALIZATION_FAILED` state. The CASA (Cluster and Slice Administration) API showed:

```
# bash on any Linux client – CASA status is reachable on the VCF Operations VIP
curl -sk https://192.168.1.77/casa/cluster/status
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
Invoke-RestMethod -Uri 'https://192.168.1.77/casa/cluster/status' | ConvertTo-Json -Depth 10
```

```
{
  "nodes_states": [{
    "name": "vcf-ops",
    "state": "INITIALIZATION_FAILED"
  }],
  "cluster_state": null
}
```

Root Cause: The unclean shutdown left the Gemfire distributed cache and HSQLDB in an inconsistent state, preventing the cluster from initializing.

7.2 HSQLDB Reset Procedure

The HSQLDB database needed to be reset to allow re-initialization:

Step 1: SSH to VCF Operations node:

```
# Run on the appliance only (subsequent steps edit appliance files at /storage/...)
ssh root@192.168.1.77
```

Step 2: Stop all VCF Operations services:

```
# Run on the appliance only (systemctl on Photon – no Windows equivalent)
systemctl stop vmware-casa
systemctl stop vmware-vcops-watchdog
```

Step 3: Examine the HSQLDB script file:

```
# Run on the appliance only (path /storage/db/casa/... only exists on the vcf-ops VM)
cat /storage/db/casa/webapp/hsqldb/casa.db.script
```

This file contains the INSERT statements that define the cluster state. Look for the `clusterMembership` document.

Step 4: Edit the HSQLDB to clear the failed initialization state:

The `casa.db.script` file needed the `initialization_state` value changed from `FAILED` to `NONE`:

```
# Run on the appliance only (file edits in appliance paths /storage/db/casa/...)
# Backup the file first
cp /storage/db/casa/webapp/hsqldb/casa.db.script \
  /storage/db/casa/webapp/hsqldb/casa.db.script.bak

# Edit the file – change initialization_state
vi /storage/db/casa/webapp/hsqldb/casa.db.script
# Find: "initialization_state":"FAILED"
# Replace with: "initialization_state":"NONE"
```

Step 5: Clear the HSQLDB log file (forces clean state):

```
# Run on the appliance only (truncates an appliance log file)
> /storage/db/casa/webapp/hsqldb/casa.db.log
```

7.3 Admin Password Reset

The admin password hash may have become invalid after the crash. Reset it:

```
# Run on the appliance only (writes a file at /storage/vcops/user/conf/...)
# Clear the admin password hash to force re-authentication
cat > /storage/vcops/user/conf/adminuser.properties << 'EOF'
#Properties for vCops user 'admin'
username=admin
hashed_password=
EOF
```

Note: After cluster initialization, the system will regenerate the password hash from the password configured during initial

setup.

7.4 SHA1 Thumbprint Fix

The cluster initialization requires valid SSL thumbprints. After the crash, the thumbprint format needed to be SHA1 (not SHA256):

```
# Run on the appliance only (reads the cert at /storage/vcops/user/conf/ssl/cert.pem)
# Get the SHA1 thumbprint of the local certificate
THUMBPRINT=$(openssl x509 -in /storage/vcops/user/conf/ssl/cert.pem -noout -fingerprint -sha1 \
| sed 's/SHA1 Fingerprint=//')

echo "Thumbprint: $THUMBPRINT"
```

This thumbprint is used in the CASA `clusterMembership` document in the HSQLDB.

7.5 Successful Cluster Initialization

Step 1: Restart services:

```
# Run on the appliance only (systemctl on Photon)
systemctl start vmware-casa
systemctl start vmware-vcops-watchdog
```

Step 2: Wait for CASA to fully start (check logs):

```
# Run on the appliance only (log path /usr/lib/vmware-casa/... only exists there)
tail -f /usr/lib/vmware-casa/casa-webapp/logs/casa.log | grep -i 'startup\|init\|error'
```

Step 3: Trigger cluster initialization via CASA API:

```
# Run on the appliance only (CASA `cluster/init` endpoint accepts POST only via localhost)
curl -sk -X POST https://localhost/casa/cluster/init \
-H "Content-Type: application/json"
```

Step 4: Verify cluster status:

```
# Run on the appliance only (`localhost` here means the vcf-ops VM itself)
curl -sk https://localhost/casa/cluster/status
```

Expected:

```
{
  "nodes_states": [{
    "name": "vcf-ops",
    "state": "CONFIGURED"
  }],
  "cluster_state": "INITIALIZED"
}
```

Step 5: Verify the slice is online:

```
# Run on the appliance only (`localhost` = vcf-ops VM; sysadmin endpoints reject remote)
curl -sk https://localhost/casa/sysadmin/slice/online_state
```

Expected: `"onlineState":"ONLINE"`

8. Phase 6: VCF Operations Admin Role Fix

8.1 Symptoms

After successful cluster initialization, the VCF Operations Suite-API showed both users with empty roles:

```
# bash on any Linux client – Suite-API is reachable on the VCF Operations VIP
# Get authentication token
TOKEN=$(curl -sk -X POST https://192.168.1.77/suite-api/api/auth/token/acquire \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"<your-password>","authSource":"local"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")

# List users
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Accept: application/json" \
  https://192.168.1.77/suite-api/api/auth/users | python3 -m json.tool
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$ops = '192.168.1.77'

# Get authentication token
$authBody = @{
    username = 'admin'
    password = '<your-password>'
    authSource = 'local'
} | ConvertTo-Json
$auth = Invoke-RestMethod -Method Post `
  -Uri "https://$ops/suite-api/api/auth/token/acquire" `
  -ContentType 'application/json' -Body $authBody
$TOKEN = $auth.token

# Note the unusual auth scheme: `vRealizeOpsToken <token>` – NOT `Bearer`
$ops_h = @{
    Authorization = "vRealizeOpsToken $TOKEN"
    Accept = 'application/json'
}

# List users
Invoke-RestMethod -Uri "https://$ops/suite-api/api/auth/users" -Headers $ops_h |
  ConvertTo-Json -Depth 10
```

Result showed:

```
{
  "users": [
```

```
{
  "username": "admin", "roleNames": []},
  {"username": "administrator@vsphere.local", "roleNames": []}
}
```

Important: VCF Operations Suite-API uses the auth header format `vRealizeOpsToken <token>` — not `Bearer` or `VMware`.

8.2 Investigation

Finding 1: The `admin` user is a super admin with implicit full access.

Testing confirmed the admin user could perform all administrative operations despite showing `roleNames: []`:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
# Admin can list users (admin-only operation)
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/users

# Admin can list adapters
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/adapters

# Admin can check cluster status
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/deployment/node/status
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
# Admin can list users (admin-only operation)
Invoke-RestMethod -Uri "https://$ops/suite-api/api/auth/users" -Headers $ops_h

# Admin can list adapters
Invoke-RestMethod -Uri "https://$ops/suite-api/api/adapters" -Headers $ops_h

# Admin can check cluster status
Invoke-RestMethod -Uri "https://$ops/suite-api/api/deployment/node/status" -Headers $ops_h
```

All three operations succeeded — confirming the `admin` user has **implicit full access** regardless of role assignment. This is **by design** in VCF Operations.

Finding 2: The Administrator role existed but was initially hidden.

After the Gemfire cache fully loaded (takes several minutes after cluster init), the Administrator role appeared:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/auth/roles
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
Invoke-RestMethod -Uri "https://$ops/suite-api/api/auth/roles" -Headers $ops_h |
  ConvertTo-Json -Depth 10
```

Showed roles including `Administrator` (489 privileges, system-created).

Finding 3: The super admin user cannot be modified via the standard user API.

```
# bash on any Linux client – illustrative: this call FAILS by design on the super-admin user
# This fails with "Cannot create or update super admin"
curl -sk -X PUT "https://192.168.1.77/suite-api/api/auth/users/<admin-id>" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"roleNames":["Administrator"]}'
```

PowerShell equivalent (run on your Windows workstation — this call also FAILS by design):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
$adminId = '<admin-id>'
$body = @{ roleNames = @('Administrator') } | ConvertTo-Json
try {
    Invoke-RestMethod -Method Put `
        -Uri "https://$ops/suite-api/api/auth/users/$adminId" `
        -Headers $ops_h -ContentType 'application/json' -Body $body
} catch {
    # Expected: "Cannot create or update super admin"
    $_.Exception.Message
}
```

8.3 Resolution: Role Assignment via Suite-API

The `administrator@vsphere.local` user needed the Administrator role explicitly assigned using the **user permissions endpoint** with a **single role-permission object** (not an array):

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/auth/users/2b4ea946-90ae-4fb9-af21-ac6246e04f1c/permissions" \
  -H "Authorization: vRealizeOpsToken $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "roleName": "Administrator",
    "allowAllObjects": true,
    "traversal-spec-instances": []
  }'
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
# CRITICAL: body must be a SINGLE object, NOT wrapped in a `permissions` array
$userId = '2b4ea946-90ae-4fb9-af21-ac6246e04f1c'
$permBody = @{
    roleName = 'Administrator'
    allowAllObjects = $true
    'traversal-spec-instances' = @()
} | ConvertTo-Json
Invoke-RestMethod -Method Put `
    -Uri "https://$ops/suite-api/api/auth/users/$userId/permissions" `
    -Headers $ops_h -ContentType 'application/json' -Body $permBody |
    ConvertTo-Json -Depth 10
```

Response (200 OK):

```
{
  "permissions": [{
    "roleName": "Administrator",
    "allowAllObjects": false,
    "scopeId": "c40f8335-4c40-4686-b68d-305f967094a0"
  }]
}
```

Critical Format Note: The request body must be a **single JSON object** with `roleName`, NOT wrapped in a `permissions` array. Using `{"permissions":[{"roleName":"Administrator"}]}` will fail with "Role with name: null cannot be found".

8.4 Super Admin Behavior (By Design)

User	roleNames	Actual Access	Notes
admin	[] (empty)	Full admin	Built-in super admin — implicit access
administrator@vsphere.local	["Administrator"]	Full admin	Explicitly assigned via permissions API

The built-in `admin` account in VCF Operations is a **super admin** that:

- Always shows `roleNames: []` in the API
- Cannot be modified via user or permissions endpoints
- Has implicit full access to all features
- This is normal and by design

9. Phase 7: VCF Operations Adapter Configuration

9.1 Collector VM Recovery

The Collector VM (vm-4016) was powered off during resource contention mitigation. After the other components were stable, it was powered back on:

```
# bash on any Linux client – reuses $SESSION from the vCenter auth call earlier
# Power on via vCenter API
curl -sk -X POST "https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?action=start" \
  -H "vmware-api-session-id: $SESSION"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $SESSION from the earlier vCenter auth block
Invoke-RestMethod -Method Post `
  -Uri 'https://vcenter.lab.local/api/vcenter/vm/vm-4016/power?action=start' `
  -Headers @{ 'vmware-api-session-id' = $SESSION }

# PowerCLI alternative (if Connect-VIServer is already established):
# Start-VM -VM (Get-VM -Id 'VirtualMachine-vm-4016') -Confirm:$false
```

Collector startup observations:

- Boot to SSH responsive: ~4 minutes
- Load average during startup: 15.14 on 4 vCPUs (extreme contention)
- Load stabilization: ~30 minutes
- CASA service fully initialized: ~30 minutes
- VCF Operations primary node recognizes collector: ~1 minute after CASA starts

Verify collector status from primary node:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
  https://192.168.1.77/suite-api/api/collectors
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
Invoke-RestMethod -Uri "https://$ops/suite-api/api/collectors" -Headers $ops_h |
  ConvertTo-Json -Depth 10
```

9.2 Adapter Restart Procedure

After the collector came online, adapters assigned to it showed `COLLECTOR_DOWN` status. They needed to be restarted:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
# For each adapter assigned to the collector (collectorId=2):
# Stop the adapter
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/stop" \
  -H "Authorization: vRealizeOpsToken $TOKEN"

# Start the adapter
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start" \
  -H "Authorization: vRealizeOpsToken $TOKEN"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
$adapterId = '<adapter-id>'

# Stop the adapter
Invoke-RestMethod -Method Put `
  -Uri "https://$ops/suite-api/api/adapters/$adapterId/monitoringstate/stop" `
  -Headers $ops_h

# Start the adapter
Invoke-RestMethod -Method Put `
  -Uri "https://$ops/suite-api/api/adapters/$adapterId/monitoringstate/start" `
  -Headers $ops_h
```

Adapters that needed restart (all on Collector 2):

Adapter	Kind	ID
lab	VcfAdapter	d7ff9f83-52e4-41ad-96fd-4b902ac36704
Application Monitoring	APPOSUCP	786ab66d-5b1b-4936-a919-3aaaad2e73e1
Infrastructure Health (collector)	VMWARE_INFRA_HEALTH	418b9fa6-74b8-469e-9624-79dcc66e111c
Infrastructure Management (collector)	VMWARE_INFRA_MANAGEMENT	92ea1e2b-5b48-4586-b7c3-071181bead22
VCF Operations (collector)	vCenter Operations Adapter	b203137d-d3f0-47e0-b70a-b428164d2fa5
Diagnostics (collector)	DiagnosticsAdapter	e2753e1a-c157-4f3d-b147-0f2d49dc90f3

Adapter	Kind	ID
Configuration Management (collector)	VCF_UNIFIED_CONFIG	2ecff0d2-8b58-42dd-bad6-8a7c99cd5c72
Log Assist (collector)	LogAssistAdapter	90896dc6-237c-453c-8b6a-bd4391ab7ff9

Important: After stopping and starting an adapter, you must wait for the collector to actually be responsive. Starting adapters while the collector JVM is still initializing will leave them in a `STOPPED` state.

9.3 NSX Adapter Manual Creation

The NSX adapter was never auto-created because the VCF adapter's initial auto-discovery had already run before the crash. It needed to be created manually.

Step 1: Create NSX credential:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -X POST "https://192.168.1.77/suite-api/api/credentials" \
-H "Authorization: vRealizeOpsToken $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "nsx-vip.lab.local",
  "adapterKindKey": "NSXTAdapter",
  "credentialKindKey": "NSXTCREDENTIAL",
  "fields": [
    {"name": "USERNAME", "value": "admin"},
    {"name": "PASSWORD", "value": "<your-password>"}
  ]
}'
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops from earlier Suite-API auth
# Field names MUST be USERNAME / PASSWORD (uppercase) – using `USER` fails with "USERNAME is mandatory"
$credBody = @{}
    name = 'nsx-vip.lab.local'
    adapterKindKey = 'NSXTAdapter'
    credentialKindKey = 'NSXTCREDENTIAL'
    fields = @(
        @{ name = 'USERNAME'; value = 'admin' }
        @{ name = 'PASSWORD'; value = '<your-password>' }
    )
} | ConvertTo-Json -Depth 5
$credResp = Invoke-RestMethod -Method Post `
    -Uri "https://$ops/suite-api/api/credentials" `
    -Headers $ops_h -ContentType 'application/json' -Body $credBody
$credResp | ConvertTo-Json -Depth 10
$credentialId = $credResp.id
```

Note: The credential field names are `USERNAME` and `PASSWORD` (uppercase). Using `USER` will fail with "USERNAME is mandatory".

Step 2: Create NSX adapter instance:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -X POST "https://192.168.1.77/suite-api/api/adapters" \
-H "Authorization: vRealizeOpsToken $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "nsx-vip.lab.local",
  "description": "NSX Manager",
  "adapterKindKey": "NSXTAdapter",
  "resourceIdentifiers": [
    {"name": "NSXTHOST", "value": "nsx-vip.lab.local"},
    {"name": "AUTO_DISCOVERY", "value": "true"},
    {"name": "ENABLE_ALERTS_FROM_NSX", "value": "false"},
    {"name": "VCURL", "value": "vcenter.lab.local"},
    {"name": "VMEntityVCID", "value": "92109cf0-ad3b-4ffa-8972-a77bb7fadacf"},
    {"name": "NSX_CLUSTER_ID", "value": "6c55d856-ab96-4190-8495-3cc8cb23450c"}
  ],
  "credential": {"id": "<credential-id>"},
  "collectorId": 2
}'
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops / $credentialId from earlier blocks
$adapterBody = @{
  name           = 'nsx-vip.lab.local'
  description    = 'NSX Manager'
  adapterKindKey = 'NSXTAdapter'
  resourceIdentifiers = @(
    @{ name = 'NSXTHOST'; value = 'nsx-vip.lab.local' }
    @{ name = 'AUTO_DISCOVERY'; value = 'true' }
    @{ name = 'ENABLE_ALERTS_FROM_NSX'; value = 'false' }
    @{ name = 'VCURL'; value = 'vcenter.lab.local' }
    @{ name = 'VMEntityVCID'; value = '92109cf0-ad3b-4ffa-8972-a77bb7fadacf' }
    @{ name = 'NSX_CLUSTER_ID'; value = '6c55d856-ab96-4190-8495-3cc8cb23450c' }
  )
  credential = @{ id = $credentialId } # or '<credential-id>' if running standalone
  collectorId = 2
} | ConvertTo-Json -Depth 6
$adapterResp = Invoke-RestMethod -Method Post `
  -Uri "https://$ops/suite-api/api/adapters" `
  -Headers $ops_h -ContentType 'application/json' -Body $adapterBody
$adapterResp | ConvertTo-Json -Depth 10
$adapterId = $adapterResp.id
```

Step 3: Start the adapter:

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -X PUT \
  "https://192.168.1.77/suite-api/api/adapters/<adapter-id>/monitoringstate/start" \
  -H "Authorization: vRealizeOpsToken $TOKEN"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops / $adapterId from earlier blocks
Invoke-RestMethod -Method Put `
  -Uri "https://$ops/suite-api/api/adapters/$adapterId/monitoringstate/start" `
  -Headers $ops_h
```

Step 4: Verify collection (within 60 seconds):

```
# bash on any Linux client – reuses $TOKEN from the Suite-API auth above
curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" \
```

```
"https://192.168.1.77/suite-api/api/adapters/<adapter-id>"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation – reuses $ops_h / $ops / $adapterId from earlier blocks
$adapter = Invoke-RestMethod -Uri "https://$ops/suite-api/api/adapters/$adapterId" -Headers $ops_h
$adapter | ConvertTo-Json -Depth 10
"numberOfResourcesCollected: $($adapter.numberOfResourcesCollected)"
```

Expected: `numberOfResourcesCollected > 0`

9.4 Final Adapter Status

Adapter	Status	Health	Resources
vcenter (VMWARE)	DATA_RECEIVING	GREEN	33
nsx-vip.lab.local (NSXAdapter)	DATA_RECEIVING	GREEN	1+
lab (VcfAdapter)	DATA_RECEIVING	ORANGE	2
Container	DATA_RECEIVING	GREEN	43
VCF Operations API (vcf-ops)	DATA_RECEIVING	GREEN	1
VCF Operations Adapter (vcf-ops)	DATA_RECEIVING	GREEN	13
VCF Operations Adapter (collector)	DATA_RECEIVING	GREEN	7
Infrastructure Health (vcf-ops)	DATA_RECEIVING	GREEN	59
Infrastructure Health (collector)	DATA_RECEIVING	GREEN	3
Infrastructure Management (vcf-ops)	DATA_RECEIVING	GREEN	5
Infrastructure Management (collector)	DATA_RECEIVING	GREEN	7
Configuration Management (collector)	DATA_RECEIVING	GREEN	0
Diagnostics (vcf-ops)	DATA_RECEIVING	GREEN	7
Diagnostics (collector)	DATA_RECEIVING	GREEN	2
Application Monitoring (collector)	DATA_RECEIVING	GREEN	1
Log Assist (collector)	ERROR	ORANGE	1

Note: Log Assist adapter shows ERROR because the Logs VM was powered off. This is expected and resolves when the Logs VM is powered back on.

10. Key Learnings and Reference

10.1 Critical API Endpoints Used

vCenter REST API

```
# Reference listing (HTTP method/URL only – not paste-ready)
# Session creation
POST https://vcenter.lab.local/api/session
Header: Authorization: Basic <base64>

# VM power operations
POST https://vcenter.lab.local/api/vcenter/vm/{vm-id}/power?action=start|stop|reset
Header: vmware-api-session-id: <session>

# VM details
GET https://vcenter.lab.local/api/vcenter/vm/{vm-id}
```

PowerShell: See the paste-ready PS sibling under section 5.2 (vCenter session + VM reset) and 6.3 (VM power-off) for the equivalent `Invoke-RestMethod` calls.

SDDC Manager REST API

```
# Reference listing (HTTP method/URL only – not paste-ready)
# Authentication
POST https://sddc-manager.lab.local/v1/tokens
Body: {"username":"administrator@vsphere.local","password":"<your-password>"}

# Task retry (key for recovery)
PATCH https://sddc-manager.lab.local/v1/tasks/{task-id}
Body: {"status":"IN_PROGRESS"}

# Management components status
GET https://sddc-manager.lab.local/v1/vcf-management-components
```

PowerShell: See the paste-ready PS siblings in sections 5.4 (auth), 6.5 (task retry PATCH), and 6.5 verification (mgmt-components GET) for `Invoke-RestMethod` equivalents.

VCF Operations Suite-API

```
# Reference listing (HTTP method/URL only – not paste-ready)
# Authentication (note: different auth header format)
POST https://192.168.1.77/suite-api/api/auth/token/acquire
Body: {"username":"admin","password":"<your-password>","authSource":"local"}
Header format: Authorization: vRealizeOpsToken <token>

# User management
GET /suite-api/api/auth/users
GET /suite-api/api/auth/users/{id}
PUT /suite-api/api/auth/users/{id}/permissions

# Adapter management
GET /suite-api/api/adapters
POST /suite-api/api/adapters
PUT /suite-api/api/adapters/{id}/monitoringstate/start
PUT /suite-api/api/adapters/{id}/monitoringstate/stop

# Credential management
```

```
POST /suite-api/api/credentials

# Cluster and system
GET /suite-api/api/collectors
GET /suite-api/api/deployment/node/status
GET /suite-api/api/auth/roles
```

PowerShell: See the paste-ready PS siblings in sections 8.1 (auth + list users), 8.2 (admin GETs + roles), 8.3 (PUT permissions), and 9.1-9.3 (collectors GET, adapter start/stop, credential POST, adapter POST) for the `Invoke-RestMethod` equivalents. Auth header is `vRealizeOpsToken $TOKEN`, NOT `Bearer`.

VCF Operations CASA API (Internal)

```
# Run on the appliance only - `https://localhost/casa/...` only resolves on the vcf-ops VM itself
# (sysadmin/init endpoints reject remote access)
# Cluster status (no auth required from localhost)
GET https://localhost/casa/cluster/status
GET https://localhost/casa/sysadmin/slice/online_state

# Cluster initialization
POST https://localhost/casa/cluster/init
```

PowerShell: The `cluster/status` endpoint is also reachable on the VIP — see section 7.1 for the `Invoke-RestMethod` example against `https://192.168.1.77/casa/cluster/status`. The `sysadmin/*` and `cluster/init` endpoints have no Windows-runnable equivalent; they must be invoked from the appliance shell.

Fleet (vRSLCM) API

```
# Reference listing (HTTP method/URL only – not paste-ready)
# Authentication
POST https://fleet.lab.local:8080/lcm/authzn/api/login
Body: {"username":"admin@local","password":"<your-password>"}

# Environment status
GET /lcm/lcops/api/v2/environments/{env-id}

# Request status
GET /lcm/lcops/api/v2/requests/{request-id}
```

PowerShell: See the paste-ready PS sibling in section 6.4 (Fleet auth + environment status) for the `Invoke-RestMethod` / `Invoke-WebRequest` equivalent.

Fleet PostgreSQL Database

```
# Run on the appliance only (psql + sudo only exist on Fleet's Photon shell – no client equivalent)
# Connect to Fleet database
ssh root@192.168.1.78
sudo -u postgres psql -d vrlcm

# Key tables
SELECT * FROM vm_rs_request WHERE id = '<request-id>';
SELECT * FROM vm_lcops_environments WHERE id = '<env-id>';
```

10.2 Service Startup Times Under Load

In a nested lab environment with resource contention, Java-based services take significantly longer to start:

Service	Normal Startup	Under Load (Nested)	Port
SDDC Manager domainmanager	2-3 min	37 min	7200 (HTTP)
SDDC Manager operationsmanager	2-3 min	30 min	7300
Fleet LCM backend	3-5 min	48 min	8080
VCF Operations CASA	2-3 min	10-15 min	443
VCF Operations Collector CASA	2-3 min	5-10 min	443
NSX Manager services	3-5 min	10-15 min	443

Rule of Thumb: In nested environments, expect startup times to be **5-10x longer** than normal. Do not assume a service has failed — check CPU load and be patient.

10.3 Common Pitfalls

1. SSH ! Character in Passwords

When using SSH/Paramiko to execute commands containing `!` in passwords, bash interprets `!` as history expansion. **Use heredocs:**

```
# bash language illustration – the `!` quoting issue is bash-specific
python3 << 'PYEOF'
# Code with passwords containing ! characters
password = "<your-password>"
PYEOF
```

PowerShell equivalent (Windows has the opposite-direction quirk):

```
# PowerShell: `!` is harmless in double-quoted strings, but `$` triggers variable interpolation.
# To embed a literal `$`, use a single-quoted string OR escape with backtick.
$password = '<your-password>' # single quotes = literal, no interpolation
# Equivalent here-string for multi-line literal content (closing '@ MUST be at column 0):
$payload = @'
{"password":"<your-password>","note":"$ and ! are both literal in single-quoted here-strings"}
'@
```

2. VCF Operations Auth Header

VCF Operations Suite-API uses `vRealizeOpsToken` prefix, NOT `Bearer` :

```
Authorization: vRealizeOpsToken abc123...
```

3. SDDC Manager HTTP vs HTTPS Internal Ports

The `domainmanager` service on port 7200 uses **HTTP** (not HTTPS):

```
# Run on the appliance only (`localhost` here means the SDDC Manager VM)
# WRONG - will fail with "wrong version number"
curl -sk https://localhost:7200/health

# CORRECT
curl -sk http://localhost:7200/health
```

PowerShell note: Same scheme rule applies if you tunnel SSH-port-forwarded — `Invoke-RestMethod -Uri 'http://127.0.0.1:7200/health'` (NOT `https://`). The "wrong version number" error means TLS handshake against a plaintext listener.

4. VCF Operations Permissions API Format

The PUT body for user permissions must be a **single object**, not wrapped in an array:

```
// CORRECT
{"roleName": "Administrator", "allowAllObjects": true}

// WRONG - will get "Role with name: null"
{"permissions": [{"roleName": "Administrator"}]}
```

5. NSX Adapter Credential Fields

NSX credential fields must use `USERNAME` and `PASSWORD` (uppercase):

```
{"name": "USERNAME", "value": "admin"}
// NOT: {"name": "USER", "value": "admin"}
```

6. CPU Soft Lockups in Nested VMs

When a VM console shows `soft lockup`, it means the CPU is completely saturated. The VM cannot be accessed via SSH. Options: 1. Wait (may resolve once other VMs finish starting) 2. Power off non-essential VMs to free resources 3. Hard reset via vCenter API as last resort

7. Gemfire Cache Loading Time

After VCF Operations cluster initialization, the Gemfire distributed cache takes several minutes to fully populate. Roles, users, and other data may not appear in API responses immediately. Wait 5-10 minutes before concluding data is missing.

11. Document Information

Field	Value
Document Title	VCF 9.0 Lab Disaster Recovery — Windows Update Crash
Version	1.1

Field	Value
Author	Virtual Control LLC
Date Created	March 15, 2026
Last Updated	May 7, 2026
Environment	VCF 9.0.2 / Nested Lab / VMware Workstation 17
Incident Date	March 13-15, 2026
Recovery Duration	~48 hours (across multiple sessions)

Revision History

Version	Date	Notes
1.0	March 15, 2026	Initial release.
1.1	May 7, 2026	Option B PowerShell-sibling pass: every translatable bash block (vCenter REST, SDDC Manager REST, Fleet API, VCF Operations Suite-API, CASA on the VIP) now has a paste-ready <code>Invoke-RestMethod</code> PowerShell sibling underneath. Appliance-only blocks (ESXi <code>esxccli / esxcfg-vswitch / vmkping</code> , NSX admin CLI <code>restart service / get , psql , systemctl , vi / cp on /storage/... , localhost CASA sysadmin / init endpoints</code>) flagged with "Run on the appliance only" comments — no PS sibling. Added top-of-doc callout linking to POWERSHELL-TRANSLATION-REFERENCE.md. Section 10 reference listings annotated with cross-references to translated examples. Uncertainty flagged on Fleet auth-token return path (header vs body varies by version).

This document is part of the VCF 9.0 Lab Documentation Suite: - **VCF9-Master-Bible.pdf** — Complete VCF 9.0 reference guide - **VCF-Troubleshooting-Handbook.pdf** — General troubleshooting procedures - **VCF-vSAN-Troubleshooting-Handbook.md** — vSAN-specific troubleshooting - **VCF-Operations-Configuration-Handbook.pdf** — VCF Operations setup guide - **VCF-Windows-Update-Crash-Recovery.md** — This document